



Task 3: System Auditing, Log Analysis & Hardening

Prepared by: Fatimah Adnan

Date: May 2026

Internship ID: APSPL2631283

Timeline: Days 25–36

1. Executive Summary

The objective of this task was to identify, exploit, and mitigate critical web vulnerabilities listed in the OWASP Top 10. Using the Damn Vulnerable Web Application (DVWA) environment, I performed manual exploitation of SQL Injection, XSS, and CSRF. This report documents the transition from offensive exploitation to defensive coding and the implementation of robust web security headers.

2. Injection & Broken Access Control

I utilized DVWA to demonstrate how poorly sanitized inputs lead to database exposure and unauthorized file access.

- **Step 1: SQL Injection (SQLi)**

I injected malicious SQL queries into the user ID field to bypass authentication and dump the backend database.

Command/Payload: ' OR '1'='1

Findings: Successfully extracted the entire user table, including plain-text usernames and hashed passwords.

Prevention: Reconfigured the source code to use **Prepared Statements** (Parameterized Queries), ensuring input is treated as data rather than executable code.

- **Step 2: File Inclusion (LFI/RFI)**

I tested the application for improper path handling to access sensitive local files.

Payload: ../../../../etc/passwd

Result: Verified Local File Inclusion (LFI) by reading the system's password file. I further documented the risks of Remote File Inclusion (RFI) where external malicious scripts can be executed on the server.

3. Cross-Site Scripting (XSS) & Request Forgery (CSRF)

I explored client-side attacks that target the end-user's session and browser.

- **Step 1: Stored and Reflected XSS**

I injected JavaScript payloads into comment fields (Stored) and URL parameters (Reflected) to hijack session cookies.

Payload: `<script>alert(document.cookie)</script>`

Mitigation: Implemented strict **Input Validation** and a **Content Security Policy (CSP)** to restrict the execution of unauthorized inline scripts.

- **Step 2: Cross-Site Request Forgery (CSRF)**

I crafted a malicious HTML page that, when visited by a logged-in user, automatically sent a request to change their DVWA password without their consent.

Defense: Demonstrated the implementation of **Anti-CSRF Tokens**, ensuring that every state-changing request requires a unique, server-validated secret.

4. Advanced Interception & Fuzzing (Burp Suite)

Using Burp Suite as a proxy, I analyzed the communication between the browser and the web server to identify hidden vulnerabilities.

- **Step 1: Request Interception**

I intercepted login POST requests to view hidden form fields and modify data in transit before it reached the server.

- **Step 2: Automated Fuzzing (Intruder)**

I utilized the **Burp Intruder** tool to perform a dictionary attack on the login page, identifying weak credentials by analyzing variations in HTTP response lengths.

5. Web Security Headers & Server Hardening

To finalize the defensive posture, I audited the HTTP response headers to prevent common browser-based attacks.

- **Step 1: Security Header Analysis**

I used *securityheaders.com* to analyze the test environment, which initially received a failing grade due to missing protections.

- **Step 2: Apache Configuration**

I modified the Apache `.htaccess` and configuration files to add critical security headers.

Headers Added: `X-Frame-Options: DENY` (prevents Clickjacking) and `Strict-Transport-Security` (enforces HTTPS).

6. Conclusion

The web security phase for Days 25–36 is complete. I have successfully demonstrated the full lifecycle of a web attack—from initial discovery with Burp Suite to final mitigation through secure coding and header configuration. All exploit screenshots, mitigation notes, and the 8-minute video demonstration have been uploaded to the GitHub repository.
